

The Representation Technique Speeds Up Subset Sum Only on Random Inputs

Le Duc Minh
minh.leduc.0210@gmail.com

Abstract—Meet-in-the-middle solves subset sum in about $2^{n/2}$ steps on every input. On random inputs, a family of methods called the *representation technique* does much better, about $2^{0.291n}$. This note explains the technique to readers who know meet-in-the-middle, and asks why its speed has not been carried over to every input. We work through a small example, derive the basic cost formula, and recompute the published exponents of Howgrave-Graham and Joux (0.337) and of Becker, Coron and Joux (0.291) from their cost models. We then report two small experiments that rule out two natural explanations for the gap between random and worst-case inputs. Finally we prove that the three coefficient sets left open by the recent worst-case work of Randolph and Węgrzycki, namely $\{0, 1\}$, $\{-1, 1\}$ and $\{-2, -1, 1, 2\}$, cannot be written as a sum of two sets in which some coefficient appears twice, which is the property their method relies on. Plain subset sum is the case $\{0, 1\}$. We do not claim a faster worst-case algorithm.

Index Terms—subset sum, knapsack, representation technique, meet-in-the-middle, worst-case analysis

I. INTRODUCTION

Given integers $a_1, \dots, a_n$ and a target b , subset sum asks whether some of the a_i add up to b . The standard fast method, meet-in-the-middle of Horowitz and Sahni [1], splits the numbers into a left and a right half, lists the $2^{n/2}$ subset sums of each half, and looks for one sum from each side that together give b . It takes about $2^{n/2}$ steps on every input. Schroeppe and Shamir [2] reduced the memory to about $2^{n/4}$, and Chen, Jin, Randolph and Servedio [5] removed a polynomial factor from the time, but no method is known that beats $2^{(1/2-\varepsilon)n}$ on every input.

In 2010 Howgrave-Graham and Joux [3] found a different way to cut the search. Instead of fixing one split of the *positions* into left and right, they split the *answer* into two smaller pieces, and use the fact that an answer can be split in a huge number of ways. On random inputs whose numbers have about n bits each, their method runs in about $2^{0.337n}$ steps. Becker, Coron and Joux [4] pushed this to about $2^{0.291n}$. Both analyses assume that the input behaves like a random one. For a carefully chosen input this can fail, so neither result gives a bound that holds for every input.

Randolph and Węgrzycki [6] recently made a version of the technique work on every input for a large class of related problems. Three cases resisted, and plain subset sum is one of them.

A. What this note does

- Section II collects the notation and the few facts about counting and modular arithmetic that the rest uses.

- Section III explains the representation idea with a small example and derives its basic cost.
- Section V recomputes the published exponents of [3] and [4] from their cost models.
- Section VI explains why these results are average-case, and reports two experiments that rule out two possible reasons for the gap.
- Section VII explains the worst-case method of [6] and proves that it has nothing to work with on the three open cases.

Derivations and proofs are in the appendices.

II. PRELIMINARIES

A. Solutions as 0/1 vectors

A choice of numbers is a vector $x \in \{0, 1\}^n$, where $x_i = 1$ means that a_i is chosen. Its sum is $a \cdot x = \sum_i a_i x_i$, and its *weight* $|x|$ is the number of chosen items. We write ℓ for the weight of a solution and $\lambda = \ell/n$ for its fraction of the items. On random hard inputs a solution usually has $\lambda \approx 1/2$.

B. Counting with entropy

The number of ways to choose k items out of n is $\binom{n}{k}$. For $k = pn$ there is a convenient estimate:

$$\binom{n}{pn} = 2^{H(p)n} \text{ up to a polynomial factor,}$$

where $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ is the binary entropy. For example $H(1/2) = 1$ and $H(1/4) \approx 0.811$, so there are about $2^{0.811n}$ subsets with $n/4$ items. We use $\mathcal{O}^*(\cdot)$ and “about” to mean “up to polynomial factors in n ”.

C. Residues and filters

For a positive integer M , the *residue* of an integer s is $s \bmod M \in \{0, \dots, M-1\}$. A *filter* keeps only the vectors whose sum has one chosen residue R . If the sums are spread evenly over the M residues, a filter keeps about a $1/M$ fraction of them. Residues add: if $a \cdot y \equiv R$ and $a \cdot (y+z) \equiv b$, then $a \cdot z \equiv b - R \pmod{M}$.

D. Average case and worst case

A *worst-case* bound holds for every input. An *average-case* bound holds for inputs drawn at random, often with some failure probability, and often only under an extra *heuristic* assumption that is plausible for random inputs but not proved. The hard random inputs in this area have *density* $\delta = n / \log_2 \max_i a_i$ close to 1, so the numbers have about n

bits. Smaller numbers make the problem easier for table-based methods, and much larger numbers make solutions rare.

E. Coefficient sets

Subset sum is one member of a larger family. Fix a finite set C of integers, the *coefficient set*. The task is to find $c \in C^n$ with $\sum_i c_i a_i$ equal to the target, with $c \neq 0$ when the target is 0. Subset sum is $C = \{0, 1\}$. Partition, which asks to split the numbers into two groups of equal sum, is $C = \{-1, 1\}$ with target 0. Equal subset sum, which asks for two different subsets with the same sum, is $C = \{-1, 0, 1\}$ with target 0. We write $[\pm d] = \{-d, \dots, -1, 1, \dots, d\}$ and $[-d : d] = \{-d, \dots, d\}$.

III. THE REPRESENTATION IDEA

A. Splitting the answer instead of the positions

Meet-in-the-middle fixes one division of the positions and needs every solution to split along it. The representation technique instead writes a solution x of weight ℓ as

$$x = y + z, \quad y, z \in \{0, 1\}^n, \quad |y| = |z| = \ell/2,$$

where y and z have no position in common. Any half of the ℓ chosen items can go into y , so one solution has

$$N = \binom{\ell}{\ell/2} \approx 2^\ell$$

such *representations*. Most of them are redundant: finding any one is enough. So we can afford to throw most of them away with a filter. With a modulus $M \approx N$ and a random residue R , we keep only the vectors y with $a \cdot y \equiv R \pmod{M}$. About one representation of the solution survives, while the list of candidate vectors shrinks by a factor of M .

B. A small example

Take $a = (3, 5, 8, 13, 21, 34)$ and $b = 66$. One solution is $3 + 8 + 21 + 34$, of weight $\ell = 4$, so it has $\binom{4}{2} = 6$ representations as a pair of disjoint 2-item sets. Without a filter, the candidates for y are all $\binom{6}{2} = 15$ pairs of numbers.

Now choose $M = 6$ and $R = 0$. Since $66 \equiv 0 \pmod{6}$, both y and z must have sums divisible by 6. Only three pairs pass: $\{3, 21\}$ with sum 24, $\{5, 13\}$ with sum 18, and $\{8, 34\}$ with sum 42. Matching sums against b : $24 + 42 = 66$, and $\{3, 21\}$ and $\{8, 34\}$ share no position, so they form the solution. The candidate list shrank from 15 to 3. Of the six representations of the solution, two survived the filter ($y = \{3, 21\}$ and $y = \{8, 34\}$), close to the $6/6 = 1$ we expect on average.

C. The basic algorithm

Algorithm 1 states the two-way search in general. It assumes the weight ℓ of a solution is known; in practice one tries every ℓ .

Proposition 1 (Heuristic list size). *Assume that the sums of weight- $\ell/2$ vectors are spread evenly over the residues modulo M . Then the lists Y and Z in Algorithm 1 have expected size*

$$\frac{\binom{n}{\ell/2}}{\binom{\ell}{\ell/2}} \approx D(\lambda)^n, \quad \log_2 D(\lambda) = H(\lambda/2) - \lambda,$$

Algorithm 1 Two-way representation search

Require: numbers $a_1, \dots, a_n$; target b ; solution weight ℓ

Ensure: a solution x , or $\perp$

```

1:  $M \leftarrow \binom{\ell}{\ell/2}$ ; pick  $R$  uniformly from  $\{0, \dots, M-1\}$ 
2:  $Y \leftarrow \{y : |y| = \ell/2, a \cdot y \equiv R \pmod{M}\}$ 
3:  $Z \leftarrow \{z : |z| = \ell/2, a \cdot z \equiv b - R \pmod{M}\}$ 
4: sort  $Z$  by sum
5: for  $y \in Y$  do
6:   for  $z \in Z$  with  $a \cdot z = b - a \cdot y$  do  $\triangleright$  binary search
7:     if  $y$  and  $z$  share no position then
8:       return  $y + z$ 
9:     end if
10:  end for
11: end for
12: return  $\perp$   $\triangleright$  repeat with a new  $R$ 
```

and the expected number of representations of a fixed solution that survive the filter is 1. For $\lambda = 1/2$ this gives $D(1/2)^n \approx 2^{0.3113n}$.

The derivation is in Appendix A. The formula is the one given by Howgrave-Graham and Joux [3, Sec. 4.1] in a different but equivalent form. Figure 1 plots it.

The proposition is about list *sizes*. The hard part is to *build* Y in time close to its size. Listing all $\binom{n}{n/4} \approx 2^{0.811n}$ weight- $n/4$ vectors and filtering them would cost far more than $2^{n/2}$. The published algorithms build the filtered lists by applying the same idea again one level down, as the next section explains.

IV. PUBLISHED REFINEMENTS

V. PUBLISHED REFINEMENTS

A. Four pieces instead of two (HGJ)

Howgrave-Graham and Joux [3] split a solution of weight $n/2$ into *four* pieces of weight $n/8$, $x = x_1 + x_2 + x_3 + x_4$. They first match pairs of pieces under one modulus, then match the two pair-sums exactly. Each four-way list is small enough to build with an unbalanced form of the Schroeppe–Shamir algorithm. With the top modulus written as $M = 2^{\beta n}$ for some $1/4 < \beta < 1/3$, their analysis gives

$$T(\beta) = \max(2^{0.3113n}, 2^{(0.0872+\beta)n}), \quad \text{memory} = 2^{(0.5436-\beta)n}. \quad (1)$$

The best time is at $\beta \rightarrow 1/4$, giving $2^{0.337n}$ with $2^{0.294n}$ memory. The first term is the list-size bound of Proposition 1. The second term is the cost of merging two lists of size L under a modulus M , which produces about L^2/M pairs before the filter; leaving this term out would give the lower figure $2^{0.3113n}$. Larger β trades time for memory: $\beta \approx 0.2936$ gives memory $2^{n/4}$ at time $2^{0.381n}$, and $\beta \rightarrow 1/3$ gives memory $2^{0.211n}$ at time $2^{0.421n}$.

B. Allowing -1 in the pieces (BCJ)

Becker, Coron and Joux [4] let the pieces use the coefficients $\{-1, 0, 1\}$ instead of $\{0, 1\}$. A chosen item can now be split

TABLE I
EXPONENTS RECOMPUTED FROM THE PUBLISHED COST MODELS OF [3],
[4]. AN EXPONENT e MEANS A RUNNING TIME OF ABOUT 2^{en} .

Quantity	Exponent
Two-way list size $D(1/2)$ (Prop. 1)	0.3113
HGJ four-way algorithm, best β	0.337
BCJ model with $\alpha = \beta = \gamma = 0$	0.337
BCJ model, minimised	0.291
Meet-in-the-middle, for comparison	0.5

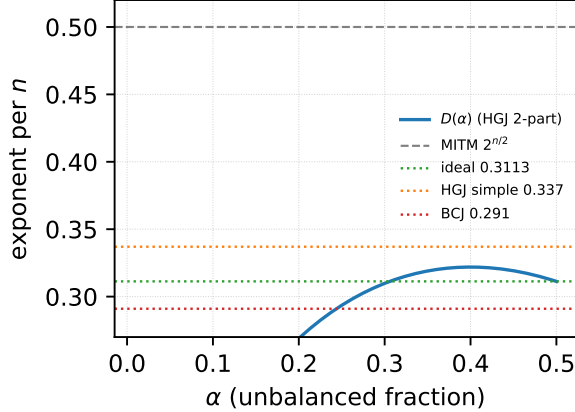


Fig. 1. The two-way list-size exponent $\log_2 D$ of Proposition 1 as a function of the solution weight fraction (horizontal axis), with the meet-in-the-middle exponent 0.5 and the published values 0.3113, 0.337 and 0.291 as reference lines.

as $1 = 1 + 0$ or $0 + 1$, as before, but an unchosen item can also be split as $0 = 1 + (-1)$ or $(-1) + 1$. This adds many more representations, so the filters can be stronger and the lists smaller. Their algorithm uses three levels of splitting, with three parameters α, β, γ that control how many extra ± 1 pairs each level adds. Appendix B gives the full cost model. Minimising it on a grid of step 0.0005 gives time $2^{0.2911n}$ at $(\alpha, \beta, \gamma) = (0.027, 0.017, 0.003)$. This matches the published optimum $2^{0.291n}$ at $(0.0267, 0.0168, 0.0029)$. Setting $\alpha = \beta = \gamma = 0$, so that no extra ± 1 pairs are allowed, gives back $2^{0.337n}$, the HGJ value. Table I summarises the numbers.

Bonnetain, Bricout, Schrottenloher and Shen [7] later reached $2^{0.283n}$ with pieces that use four values. All of these are average-case, heuristic bounds.

VI. WHY ONLY RANDOM INPUTS?

A. The assumption behind the bounds

Every bound above rests on one assumption: the sums of the representations of a solution are spread fairly evenly over the residues modulo M . Then a random residue R catches about one representation, and the filtered lists have the sizes in Proposition 1. For random numbers this is very plausible. For a chosen input it can fail badly. If every a_i is a multiple of M , for example, every sum has residue 0. A filter with $R \neq 0$ then keeps nothing, and a filter with $R = 0$ keeps everything,

so the filter never shrinks the search. A worst-case algorithm has to cope with inputs like this.

A natural guess is that some recognisable kind of input is responsible for the gap. We tested two such guesses with small experiments on six input families:

- *random*: random numbers of about n bits (density 1);
- *dissociated*: numbers whose 2^n subset sums are all different;
- *geometric* and *near-geometric*: powers of two, exactly or with small random changes;
- *arithmetic*: numbers in an arithmetic progression;
- *concentrated*: all numbers multiples of the modulus M .

Each input has a planted solution of weight $n/2$. Experiment 2 also uses a seventh family, *dense random*, with small random numbers.

B. Experiment 1: do the representations spread out?

For the planted solution we computed the residues, modulo $M \approx N$, of all its $N = \binom{n/2}{n/4}$ two-way representations, and measured the *coverage*: the fraction of the M residues that receive at least one representation. If N items land in N residues at random, the expected coverage is $1 - 1/e \approx 0.63$, so values near 0.6 mean “spread like random”.

At $n = 32$ ($N = 12,870$), the random, dissociated, geometric and near-geometric families all have coverage between 0.59 and 0.65. The arithmetic family drops to 0.009 and the concentrated family to 0.0001. At smaller n the geometric families were lower (0.33–0.38 at $n = 20$) but rose with n . So the families that look hardest keep the random-like spread. Only the families with built-in arithmetic structure lose it.

C. Experiment 2: are there too many false matches?

A filter can also fail by keeping too many *wrong* candidates. For each input we counted the subsets that reach the target: T with exactly $n/2$ items (true solutions of the expected weight) and P with fewer items (other subsets that also hit the target and compete in the search). For $n = 12, 14, \dots, 20$, the dissociated, geometric and near-geometric families always had $T = 1$ and $P = 0$, and the random family had $P = 0$ except once ($P = 1$ at $n = 14$). Large values of P appeared only in the arithmetic and dense random families, which have many solutions anyway.

D. What the experiments say

Neither guess explains the gap. The inputs that look hardest behave like random ones in both tests, and the inputs that break the filter are highly structured. A worst-case input that defeats the technique would have to be unstructured enough to resist other methods and still concentrate its residues, and we did not find such a family. These are small experiments, so they rule out explanations, not algorithms.

VII. THE WORST-CASE METHOD AND THE OPEN CASES

Randolph and Węgrzycki [6] turned the representation idea into worst-case algorithms for many coefficient sets C . One

TABLE II

COMPUTER SEARCH OVER FACTORISATIONS $C = C_1 + C_2$ WITH SMALL $C_1, C_2 \subseteq [-3, 3]$. “USEFUL” COUNTS FACTORISATIONS IN WHICH SOME COEFFICIENT HAS TWO OR MORE REPRESENTATIONS.

C	useful	max. representations
$\{0, 1\}$ (subset sum)	0	1
$\{-1, 1\}$ (Partition)	0	1
$[\pm 2]$	0	1
$\{-1, 0, 1\}$ (equal subset sum)	6	2
$[\pm 3]$	4	2
$[-2 : 2]$	39	3

of their main tools is *coefficient shifting*. They write the coefficient set as a *sumset*

$$C = C_1 + C_2 = \{c_1 + c_2 : c_1 \in C_1, c_2 \in C_2\},$$

and represent each coefficient vector as a sum of a vector over C_1 and a vector over C_2 . This produces many representations only if some coefficient can be written as $c_1 + c_2$ in at least two ways. For example,

$$\{0, 1\} + \{-3, -2, 1, 2\} = [\pm 3],$$

where $-2 = 0 + (-2) = 1 + (-3)$ and $2 = 0 + 2 = 1 + 1$ each have two representations. Equal subset sum works too: $\{0, 1\} + \{-1, 0\} = \{-1, 0, 1\}$, and 0 has two representations.

Their results leave three coefficient sets open: $\{0, 1\}$ (subset sum), $\{-1, 1\}$ (Partition) and $[\pm 2] = \{-2, -1, 1, 2\}$. We say that $C = C_1 + C_2$ is a *useful* factorisation if some element of C has at least two representations $c_1 + c_2$.

Proposition 2. *None of $\{0, 1\}$, $\{-1, 1\}$ and $\{-2, -1, 1, 2\}$ has a useful factorisation.*

The proof, in Appendix C, uses only the elementary fact that $|C_1 + C_2| \geq |C_1| + |C_2| - 1$. A computer search over all small C_1, C_2 inside $[-3, 3]$ agrees (Table II). So for plain subset sum, coefficient shifting has nothing to work with. Note that $\{0, 1\}$ contains 0, so having a zero coefficient is not what matters; what matters is the size and shape of the set.

VIII. CONCLUSION

The representation technique gets its speed from a simple fact: a solution can be split in very many ways, so a random filter can throw most of them away and still keep one. On random inputs this gives about $2^{0.291n}$ steps. The bound depends on the representations being spread out modulo the filter, which is not guaranteed for every input. In our experiments, the inputs that look hardest are spread out like random ones, so the gap is not explained by an obvious class of bad inputs. The known worst-case method needs a useful factorisation of the coefficient set, and Proposition 2 shows that subset sum has none. A worst-case $2^{n/3}$ algorithm for subset sum, or anything below $2^{n/2}$, would therefore need a different source of extra representations.

APPENDIX A DERIVATION OF PROPOSITION 1

Let x be a solution of weight $\ell = \lambda n$, and let $M = \binom{\ell}{\ell/2}$.

Survivors. The representations of x are the pairs $(y, x - y)$ where y picks $\ell/2$ of the ℓ chosen positions, so there are M of them. Under the assumption, each $a \cdot y$ has residue R with probability $1/M$, so the expected number with $a \cdot y \equiv R$ is $M \cdot (1/M) = 1$. For each such y , the partner $z = x - y$ has $a \cdot z = b - a \cdot y \equiv b - R$, so it lies in Z , and the pair is found by the search.

List size. There are $\binom{n}{\ell/2}$ vectors of weight $\ell/2$. Under the assumption, about a $1/M$ fraction of them has residue R , so $|Y| \approx \binom{n}{\ell/2} / \binom{\ell}{\ell/2}$, and the same holds for Z .

Exponent. By the entropy estimate of Section II, $\log_2 \binom{n}{\ell/2} \approx nH(\lambda/2)$ and $\log_2 \binom{\lambda n}{\lambda n/2} \approx \lambda n H(1/2) = \lambda n$. So $\log_2 |Y| \approx n(H(\lambda/2) - \lambda)$, which is the claim. For $\lambda = 1/2$, $H(1/4) = 0.8113$, so the exponent is 0.3113.

Equivalent form. Expanding the entropy,

$$H(\lambda/2) = 1 - \frac{\lambda}{2} \log_2 \lambda - \frac{2-\lambda}{2} \log_2 (2-\lambda),$$

so

$$D(\lambda) = \frac{2}{\lambda^{\lambda/2} (2-\lambda)^{(2-\lambda)/2}} \cdot 2^{-\lambda},$$

which is the form in [3, Sec. 4.1]. $\square$

APPENDIX B THE BCJ COST MODEL

This appendix states the model of [4] that we evaluated. The algorithm splits the solution three times, into 2, 4 and then 8 pieces. At each level the pieces may contain extra +1 and -1 entries that cancel when the pieces are added. As fractions of n , the numbers of +1 and -1 entries in one piece at the three levels ν, κ, ω are

$$\begin{aligned} N_\nu(1) &= \frac{1}{4} + \alpha, & N_\nu(-1) &= \alpha, \\ N_\kappa(1) &= \frac{1}{8} + \frac{\alpha}{2} + \beta, & N_\kappa(-1) &= \frac{\alpha}{2} + \beta, \\ N_\omega(1) &= \frac{1}{16} + \frac{\alpha}{4} + \frac{\beta}{2} + \gamma, & N_\omega(-1) &= \frac{\alpha}{4} + \frac{\beta}{2} + \gamma. \end{aligned}$$

At each level the modulus M_χ is chosen so that about one representation of each piece survives, as in Appendix A. The list at level χ then has size L_χ equal to the number of vectors with the counts N_χ divided by the product of the moduli used so far. Let h_ω be the entropy of the coefficient distribution at the bottom level, so the bottom lists are built from half-lists of size $2^{h_\omega n/2}$. The running time is

$$T = \max \left(2^{h_\omega n/2}, L_\omega, \frac{L_\omega^2}{M_\kappa}, L_\kappa, \frac{L_\kappa^2}{M_\nu}, L_\nu, L_\nu^2 \frac{M_\nu M_\kappa M_\omega}{2^n} \right).$$

Each fraction L^2/M is the number of pairs produced when two lists of size L are merged under a modulus M . At the optimum $(\alpha, \beta, \gamma) \approx (0.027, 0.017, 0.003)$ the base-list term is $2^{0.266n}$, below the binding terms, and $T \approx 2^{0.291n}$.

APPENDIX C
PROOF OF PROPOSITION 2

Lemma 1. *For finite nonempty sets of integers A and B , $|A + B| \geq |A| + |B| - 1$.*

Proof. Write $A = \{x_1 < \dots < x_p\}$ and $B = \{y_1 < \dots < y_q\}$. The $p + q - 1$ sums

$$x_1 + y_1 < x_1 + y_2 < \dots < x_1 + y_q < x_2 + y_q < \dots < x_p + y_q$$

are strictly increasing, so they are distinct elements of $A + B$. $\square$

Let $C = C_1 + C_2$ with both sets nonempty, and suppose some element has two representations. Then the $|C_1| |C_2|$ pairs (c_1, c_2) produce fewer than $|C_1| |C_2|$ distinct sums, so $|C| < |C_1| |C_2|$. In particular both $|C_1| \geq 2$ and $|C_2| \geq 2$, because if one set has a single element, every sum has exactly one representation.

Two-element sets. Let C be $\{0, 1\}$ or $\{-1, 1\}$. By Lemma 1, $2 = |C| \geq |C_1| + |C_2| - 1 \geq 3$, a contradiction.

The set $[\pm 2]$. Here $|C| = 4$, so $|C_1| + |C_2| \leq 5$ by Lemma 1, and $|C_1| |C_2| > 4$ from above. With both sizes at least 2, the only option is sizes 2 and 3; say $C_1 = \{x_1 < x_2\}$ and $C_2 = \{y_1 < y_2 < y_3\}$. Then $|C_1 + C_2| = 4 = |C_1| + |C_2| - 1$. The two increasing chains

$$x_1 + y_1 < x_1 + y_2 < x_1 + y_3 < x_2 + y_3, \quad x_1 + y_1 < x_2 + y_1 < x_2 + y_2 < x_2 + y_3$$

each list four distinct elements of a set of size four, so both are the whole set in increasing order. Comparing them term by term gives $x_1 + y_2 = x_2 + y_1$ and $x_1 + y_3 = x_2 + y_2$. With $d = x_2 - x_1$ this means $y_2 - y_1 = y_3 - y_2 = d$, so

$$C_1 + C_2 = \{x_1 + y_1 + kd : k = 0, 1, 2, 3\},$$

an arithmetic progression. But $\{-2, -1, 1, 2\}$ is not an arithmetic progression, since its gaps are 1, 2, 1. This is a contradiction. $\square$

REFERENCES

- [1] E. Horowitz and S. Sahni, "Computing partitions with applications to the knapsack problem," *J. ACM*, vol. 21, no. 2, pp. 277–292, 1974.
- [2] R. Schroeppel and A. Shamir, "A $T = O(2^{n/2})$, $S = O(2^{n/4})$ algorithm for certain NP-complete problems," *SIAM J. Comput.*, vol. 10, no. 3, pp. 456–464, 1981.
- [3] N. Howgrave-Graham and A. Joux, "New generic algorithms for hard knapsacks," in *EUROCRYPT*, 2010; IACR ePrint 2010/189.
- [4] A. Becker, J.-S. Coron, and A. Joux, "Improved generic algorithms for hard knapsacks," in *EUROCRYPT*, 2011; IACR ePrint 2011/474.
- [5] X. Chen, Y. Jin, T. Randolph, and R. A. Servedio, "Subset sum in time $2^{n/2}/\text{poly}(n)$," in *RANDOM*, 2023; arXiv:2301.07134.
- [6] T. Randolph and K. Węgrzycki, "Beating meet-in-the-middle for subset balancing problems," arXiv:2511.10823, 2025.
- [7] X. Bonnetain, R. Briceout, A. Schrottenloher, and Y. Shen, "Improved classical and quantum algorithms for subset-sum," in *ASIACRYPT*, 2020; IACR ePrint 2020/168.